Jenkins for Network Automation: A Beginner's Lab Guide

Install Jenkins on an Ubuntu workstation, then build a pipeline that reviews and pushes configuration changes to lab switches. Every change goes through Git, a dry run and a human approval first.

Ubuntu 24.04 LTS | Jenkins LTS | Ansible | About 3 hours | No prior Jenkins experience

What you will build

You will describe the network as files in a Git repository. When the repository changes, Jenkins checks out the files, uses Ansible to show what would change on each switch, waits for your approval, and then applies the change.

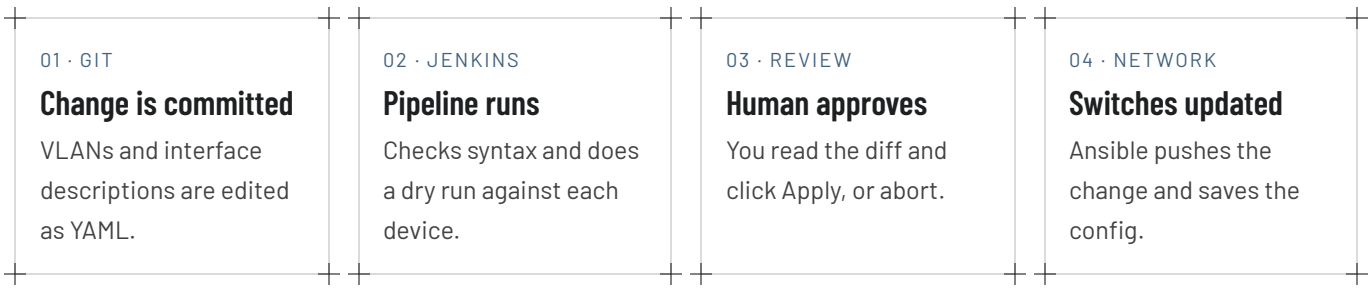


Figure 1. The change flow built in this lab.

Prerequisites

ITEM	REQUIREMENT
Workstation	Ubuntu 24.04 LTS desktop or server, 2 vCPU, 4 GB RAM, 20 GB free disk, a user with `sudo`
Network access	Internet access for packages; SSH (TCP 22) from the workstation to the lab devices
Lab devices	Two Cisco IOS or IOS-XE switches. Physical, Cisco CML, GNS3 or EVE-NG all work
Device account	A local user with privilege 15 and SSH enabled on each switch
Git repository	An empty repo on GitHub, GitLab or Gitea that the workstation can reach

Lab addressing

The commands in this guide use these addresses. Replace them with your own.

HOST	ADDRESS	ROLE
jenkins-ws	192.168.56.10	Ubuntu workstation running Jenkins
sw1	192.168.56.11	Lab access switch
sw2	192.168.56.12	Lab access switch

LAB SAFETY

Run this lab against lab equipment only. Jenkins will be able to change device configuration, so do not point it at production devices until you have the approval and rollback practices from Part 8 in place.

Part 1 · Prepare the workstation

Jenkins is a Java application. Update the system and install a supported Java runtime first.

1. Update the package index and installed packages.

```
sudo apt update
sudo apt upgrade -y
```

2. Install Java 21 and the font libraries Jenkins uses for graphs.

```
sudo apt install -y fontconfig openjdk-21-jre git python3-venv
```

3. Confirm the version. The output should start with `openjdk version "21`.

```
java -version
```

CHECK `java -version` reports 21, and `ping 192.168.56.11` gets replies.

Part 2 · Install Jenkins

Install Jenkins from the project's own LTS (long-term support) repository rather than Ubuntu's archive. This gets you current security fixes.

1. Add the Jenkins signing key and repository.

```
sudo install -d -m 0755 /etc/apt/keyrings
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" \
  | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
```

If `apt update` reports NO_PUBKEY, the key has been rotated. Copy the current key URL from the Linux install page on jenkins.io.

2. Install Jenkins, then start it and enable it at boot.

```
sudo apt update
sudo apt install -y jenkins
sudo systemctl enable --now jenkins
```

3. If the UFW firewall is on, allow the Jenkins web port (8080) from your lab subnet only.

```
sudo ufw allow from 192.168.56.0/24 to any port 8080 proto tcp
sudo ufw status
```

CHECK `systemctl status jenkins` shows `active (running)`.

Part 3 · Unlock and set up Jenkins

1. In a browser, open `http://192.168.56.10:8080`. Jenkins asks for an unlock password.
2. Read the one-time password from the workstation and paste it in:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

3. Choose **Install suggested plugins**. This includes Pipeline, Git and Credentials, which this lab needs.
4. Create your admin user. Don't keep using the default `admin` account.
5. On **Instance Configuration**, set the Jenkins URL to `http://192.168.56.10:8080/`.
6. Go to **Manage Jenkins → Security**. Check that **Jenkins' own user database** is selected and that anonymous users have no access.

Part 4 · Prepare Jenkins for network automation

4.1 Install Ansible for the Jenkins user

Jenkins runs jobs as the `jenkins` system user. Create a Python virtual environment that belongs to that user and install Ansible and the Cisco IOS collection in it. This keeps the automation tools apart from the system Python.

```
sudo -u jenkins python3 -m venv /var/lib/jenkins/netauto
sudo -u jenkins /var/lib/jenkins/netauto/bin/pip install \
  ansible-core ansible-pylibssh paramiko
sudo -u jenkins /var/lib/jenkins/netauto/bin/ansible-galaxy \
  collection install cisco.ios
```

4.2 Trust the switches' SSH host keys

Record each switch's host key once, so Ansible can connect with host-key checking turned on.

```
sudo -u jenkins mkdir -p -m 700 /var/lib/jenkins/.ssh
ssh-keyscan 192.168.56.11 192.168.56.12 \
  | sudo -u jenkins tee -a /var/lib/jenkins/.ssh/known_hosts
```

4.3 Store the device credentials in Jenkins

Passwords never go in Git. Jenkins stores them encrypted and gives them to the job only while it runs.

1. Go to **Manage Jenkins → Credentials → System → Global credentials → Add Credentials**.
2. Kind: **Username with password**. Enter the switch username and password.
3. ID: `lab-switch-creds`. The pipeline looks up the credential by this exact ID.
4. If your Git repo is private, add a second credential for it, such as a personal access token with read access.

CHECK `sudo -u jenkins /var/lib/jenkins/netauto/bin/ansible-galaxy collection list` lists `cisco.ios`.

Part 5 · Build the automation repository

Clone your empty repository to the workstation and create the files below. Together they describe the network's intended state (the config you want) and how to apply it.

FILE	PURPOSE
`inventory.yml`	Which devices exist and how to reach them
`group_vars/switches.yml`	VLANs every switch should have
`host_vars/sw1.yml`, `sw2.yml`	Interface descriptions for each switch
`site.yml`	The Ansible playbook that applies the state
`Jenkinsfile`	The pipeline stages Jenkins runs

inventory.yml

```
all:
  children:
    switches:
      hosts:
        sw1: { ansible_host: 192.168.56.11 }
        sw2: { ansible_host: 192.168.56.12 }
      vars:
        ansible_connection: ansible.netcommon.network_cli
        ansible_network_os: cisco.ios.ios
        ansible_user: "{{ lookup('env', 'NET_USER') }}"
        ansible_password: "{{ lookup('env', 'NET_PASS') }}"
```

The username and password come from environment variables that Jenkins sets at run time.

group_vars/switches.yml

```
vlans:
  - { vlan_id: 10, name: USERS }
  - { vlan_id: 20, name: PRINTERS }
  - { vlan_id: 99, name: MGMT }
```

host_vars/sw1.yml

```
interfaces:
  - name: GigabitEthernet0/1
    description: "Desk 1A - USERS"
  - name: GigabitEthernet0/2
    description: "Printer floor 1 - PRINTERS"
```

Create `host_vars/sw2.yml` the same way, using that switch's interface names.

site.yml

```
- name: Apply VLAN and interface standards
  hosts: switches
  gather_facts: false
  tasks:
    - name: Ensure VLANs exist
      cisco.ios.ios_vlans:
        config: "{{ vlans }}"
        state: merged

    - name: Set interface descriptions
      cisco.ios.ios_interfaces:
        config: "{{ interfaces }}"
        state: merged

    - name: Save running config if anything changed
      cisco.ios.ios_config:
        save_when: modified
```

`state: merged` adds or updates the items you list and leaves everything else on the switch alone. It is the safest state to start with.

Jenkinsfile

```
pipeline {
  agent any
  options { timestamps(); disableConcurrentBuilds() }
  parameters {
    booleanParam(name: 'APPLY', defaultValue: false,
      description: 'Push changes to the switches after approval')
  }
  environment {
    ANSIBLE = '/var/lib/jenkins/netauto/bin/ansible-playbook'
  }
  stages {
    stage('Syntax check') {
      steps { sh '$ANSIBLE -i inventory.yml site.yml --syntax-check' }
    }
    stage('Dry run') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'lab-switch-creds',
            usernameVariable: 'NET_USER', passwordVariable: 'NET_PASS')]) {
          sh '$ANSIBLE -i inventory.yml site.yml --check --diff'
        }
      }
    }
    stage('Approve') {
      when { expression { params.APPLY } }
      steps { input message: 'Push these changes to the lab switches?', ok: 'Apply' }
    }
    stage('Apply') {
      when { expression { params.APPLY } }
      steps {
        withCredentials([usernamePassword(credentialsId: 'lab-switch-creds',
            usernameVariable: 'NET_USER', passwordVariable: 'NET_PASS')]) {
          sh '$ANSIBLE -i inventory.yml site.yml --diff'
        }
      }
    }
  }
}
```

Every run does the dry run. The change is applied only when APPLY is ticked and someone approves.

Commit and push the files:

```
git add .
git commit -m "Initial network state: VLANs and descriptions"
git push origin main
```

Part 6 · Create and run the pipeline job

1. From the Jenkins dashboard, choose **New Item**. Name it `network-lab-changes`, select **Pipeline**, then click **OK**.
2. Under **Pipeline**, set Definition to **Pipeline script from SCM**, SCM to **Git**, and paste the repository URL. Choose the Git credential if the repo is private.
3. Set Branch Specifier to `*/main` and Script Path to `Jenkinsfile`. Save.
4. Click **Build Now** once. The first run reads the Jenkinsfile and registers the APPLY parameter. After that, the button changes to **Build with Parameters**.
5. Open the build and select **Console Output**. In the dry-run stage, lines that start with + are commands Ansible would send.
6. Run **Build with Parameters** with APPLY ticked. When the pipeline pauses, read the diff, then click **Apply**.

CHECK On sw1, `show vlan brief` lists VLANs 10, 20 and 99, and `show interfaces description` shows the new labels. If you run the build again, the PLAY RECAP shows `changed=0`.

Part 7 · Make a change through Git

From now on, change the network by editing files in Git, not by logging in to the switches.

1. Add VLAN `30 · VOICE` to `group_vars/switches.yml`, then commit and push.
2. To have Jenkins notice commits, open the job's **Configure** page. Enable **Poll SCM** with schedule `H/5 * * * *`, which checks Git about every five minutes. If your Git server can reach Jenkins, use a webhook instead.
3. The commit starts a dry-run build. Review its diff, then start a parameterised build with APPLY ticked to push the change.

Part 8 · Habits to carry beyond the lab

PRACTICE	WHY
Back up before applying	Add a stage that runs `cisco.ios.ios_config` with `backup: true` and archives the files, so you always have a known-good config to restore.
Review changes in pull requests	Protect the main branch. A second engineer reviews the YAML before it is merged.
Limit who can approve	Set `submitter` on the `input` step to a named Jenkins group.
Change one site first	Add `--limit sw1` to apply the change to a single device before the rest.
Keep Jenkins patched	Run `sudo apt upgrade jenkins` and update plugins from **Manage Jenkins → Plugins** every month.

Troubleshooting

SYMPTOM	LIKELY CAUSE AND FIX
Page on port 8080 does not load	Jenkins is still starting, or UFW is blocking the port. Check with `journalctl -u jenkins -f` and `sudo ufw status`.
`ansible-playbook: not found`	The virtual environment is not in the expected place. Check that `/var/lib/jenkins/netauto/bin` exists and belongs to the `jenkins` user.
`couldn't resolve module/action 'cisco.ios.ios_vlans'`	The collection was installed as a different user. Rerun the `ansible-galaxy` command with `sudo -u jenkins`.
Host key verification failed	The switch's key is missing or has changed. Repeat step 4.2.
Authentication failed	The credential ID doesn't match `lab-switch-creds`, or the device account isn't privilege 15.
Build waits forever at Approve	This is expected. Open the build and click **Apply** or **Abort**.